

Module 06 — SHAP Analysis (Human Pattern)

Spec: ai_chess_coach_course_shap_human_pattern.md

Phases covered:

1. Train/compare LightGBM (baseline), XGBoost, CatBoost on Human Pattern features
2. Global SHAP importance
3. Per-class SHAP importance
4. Local `explain_prediction` + waterfall examples

Module 05 (MLflow) remains separate.

1. Setup

```
In [1]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shap

from dataset.feature_engineering import (
    resolve_model_feature_columns,
    split_features_and_target,
)
from dataset.game_splits import split_by_game_id
from explainability.human_pattern_models import (
    LABEL_ORDER,
    evaluate_human_pattern_model,
    save_human_pattern_artifacts,
    train_and_compare_human_pattern_models,
)
from explainability.shap_analysis import (
    compute_class_shap_importance,
    compute_global_shap_importance,
    explain_prediction,
    sample_rows_by_game_id,
)

pd.set_option('display.max_columns', 120)
shap.initjs()

REPO_ROOT = Path('../..').resolve()
DATASET_PATH = REPO_ROOT / 'data' / 'datasets' / 'course_training_dataset.parquet'
ARTIFACT_DIR = REPO_ROOT / 'artifacts' / 'module06'
SPLIT_SEED = 42
SHAP_SAMPLE_SIZE = 5000

print(f'Dataset : {DATASET_PATH}')
print(f'Artifacts: {ARTIFACT_DIR}')
```



Dataset : C:\Users\sergiosal\source\repos\chessinsightai\data\datasets\course_training_dataset.parquet
 Artifacts: C:\Users\sergiosal\source\repos\chessinsightai\artifacts\module06

2. Load Data and Game-Level Splits

```
In [2]: course_df = pd.read_parquet(DATASET_PATH)
human_columns = resolve_model_feature_columns(course_df, feature_set='human')

train_df, val_df, test_df = split_by_game_id(
    course_df,
    target_column='error_label',
    random_state=SPLIT_SEED,
)

X_train, y_train = split_features_and_target(
    train_df, feature_columns=human_columns, sanitize_feature_names=True
)
X_val, y_val = split_features_and_target(
    val_df, feature_columns=human_columns, sanitize_feature_names=True
)
X_test, y_test = split_features_and_target(
    test_df, feature_columns=human_columns, sanitize_feature_names=True
)

print(f'Human features: {len(human_columns)} -> sanitized: {X_train.shape[1]}')
print(f'Train/val/test: {len(X_train):,} / {len(X_val):,} / {len(X_test):,}')
```

Human features: 340 -> sanitized: 340

Train/val/test: 260,423 / 55,420 / 56,224

3. Phase 1 — Model Comparison (LightGBM vs XGBoost vs CatBoost)

```
In [4]: fitted_models, comparison_df, selected_model_name = train_and_compare_human_patterns(
    X_train,
    y_train,
    X_val,
    y_val,
    random_state=SPLIT_SEED,
)

display(comparison_df.round(4))
print(f'Selected model for SHAP: {selected_model_name}')

best_model = fitted_models[selected_model_name]
test_metrics = evaluate_human_pattern_model(selected_model_name, best_model, X_test, y_test)
print(
    f'Test balanced_accuracy={test_metrics.balanced_accuracy:.4f} '
    f'macro_f1={test_metrics.f1_macro:.4f}'
)

artifact_paths = save_human_pattern_artifacts(
    ARTIFACT_DIR,
    model=best_model,
```

```

    model_name=selected_model_name,
    feature_columns=human_columns,
    comparison_df=comparison_df,
)
artifact_paths

```

	balanced_accuracy	f1_macro
model_name		
lightgbm	0.4699	0.4108
xgboost	0.4656	0.3998
catboost	0.4553	0.3937

Selected model for SHAP: lightgbm

Test balanced_accuracy=0.4664 macro_f1=0.4065

```

Out[4]: {'model': WindowsPath('C:/Users/sergiosal/source/repos/chessinsightai/artifact
s/module06/human_model.joblib'),
        'comparison': WindowsPath('C:/Users/sergiosal/source/repos/chessinsightai/arti
facts/module06/model_comparison.json'),
        'features': WindowsPath('C:/Users/sergiosal/source/repos/chessinsightai/artifa
cts/module06/feature_columns.json'),
        'labels': WindowsPath('C:/Users/sergiosal/source/repos/chessinsightai/artifact
s/module06/label_order.json'),
        'selected': WindowsPath('C:/Users/sergiosal/source/repos/chessinsightai/artifa
cts/module06/selected_model_name.txt')}

```

4. Phase 2 — Global SHAP

```

In [5]: shap_sample_df = sample_rows_by_game_id(
        test_df,
        max_rows=SHAP_SAMPLE_SIZE,
        random_state=SPLIT_SEED,
    )
X_shap, _ = split_features_and_target(
    shap_sample_df,
    feature_columns=human_columns,
    sanitize_feature_names=True,
)

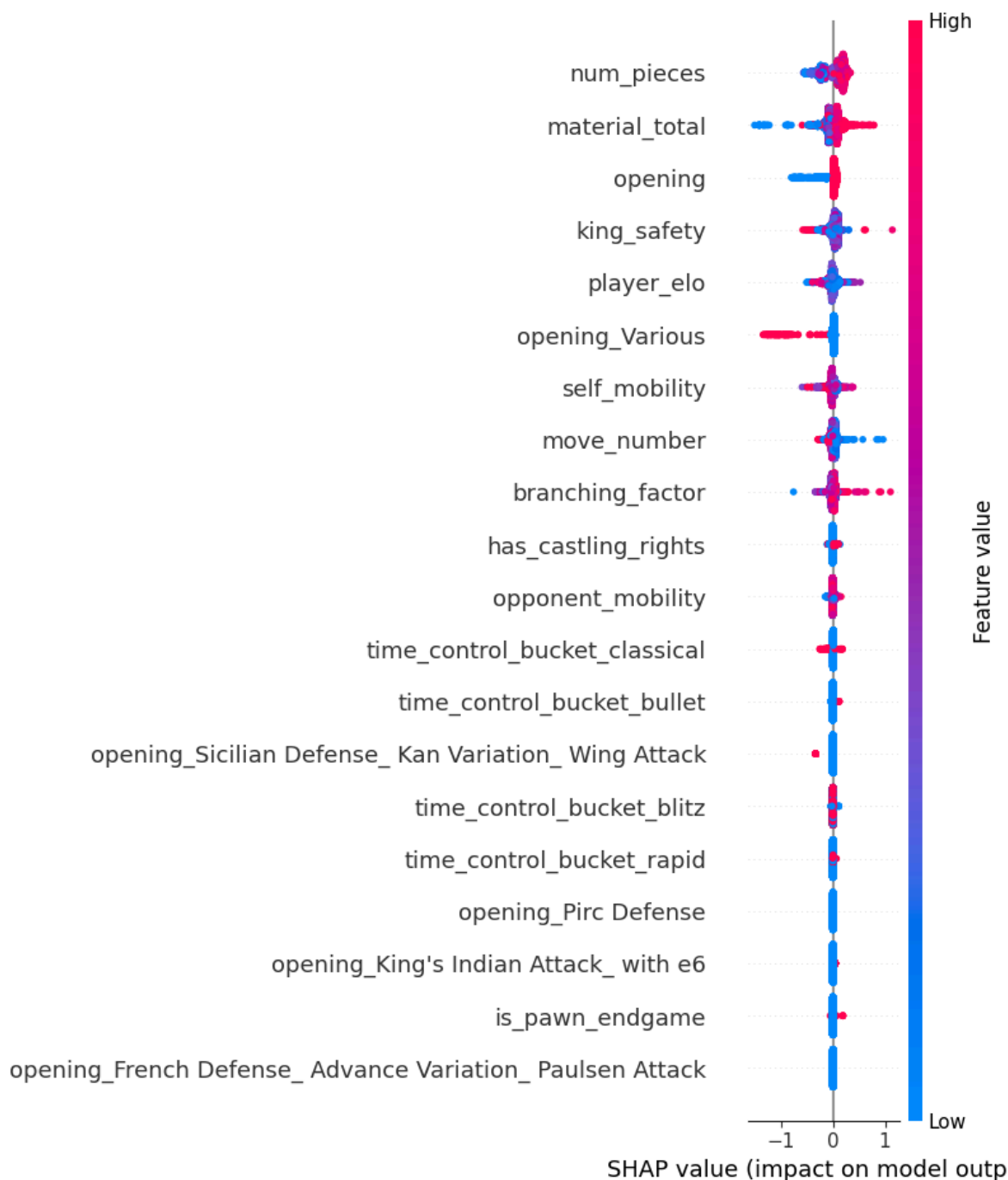
raw_importance, grouped_importance, explainer, shap_tensor = compute_global_shap(
    best_model,
    X_shap,
)

display(grouped_importance.head(20))
grouped_importance.to_parquet(ARTIFACT_DIR / 'global_shap_importance.parquet', i

shap.summary_plot(
    shap_tensor[:, :, LABEL_ORDER.index('blunder')],
    X_shap,
    max_display=20,
    show=False,
)
plt.tight_layout()
plt.show()

```

	feature	mean_abs_shap
0	material_total	0.233343
1	move_number	0.212709
2	player_elo	0.172316
3	num_pieces	0.121938
4	king_safety	0.095322
5	time_control_bucket (grouped)	0.046835
6	opening	0.045463
7	opening (grouped)	0.034689
8	has_castling_rights	0.029852
9	branching_factor	0.025061
10	opponent_mobility	0.022754
11	self_mobility	0.021052
12	is_pawn_endgame	0.000907
13	center_control	0.000000



5. Phase 3 — Per-Class SHAP

```
In [6]: class_shap_results = compute_class_shap_importance(
        shap_tensor,
        list(X_shap.columns),
        class_labels=LABEL_ORDER,
    )

    for class_label, importance_df in class_shap_results.items():
        display(importance_df.head(10))
        out_path = ARTIFACT_DIR / f'class_shap_importance_{class_label}.parquet'
        importance_df.to_parquet(out_path, index=False)
```

	feature	mean_abs_shap
0	material_total	0.535415
1	move_number	0.385568
2	player_elo	0.344760
3	num_pieces	0.180569
4	king_safety	0.166708
5	time_control_bucket (grouped)	0.092305
6	opening	0.058514
7	branching_factor	0.025383
8	opening (grouped)	0.025166
9	opponent_mobility	0.023221

	feature	mean_abs_shap
0	player_elo	0.187356
1	material_total	0.097127
2	time_control_bucket (grouped)	0.061526
3	has_castling_rights	0.058189
4	opening	0.047680
5	opening (grouped)	0.039914
6	king_safety	0.039742
7	opponent_mobility	0.039325
8	num_pieces	0.028709
9	move_number	0.025695

	feature	mean_abs_shap
0	move_number	0.400819
1	material_total	0.194772
2	king_safety	0.108483
3	num_pieces	0.101161
4	player_elo	0.094178
5	branching_factor	0.024927
6	has_castling_rights	0.022638
7	self_mobility	0.020554
8	opponent_mobility	0.015518
9	time_control_bucket (grouped)	0.013269

	feature	mean_abs_shap
0	num_pieces	0.177312
1	material_total	0.106058
2	opening	0.069925
3	king_safety	0.066352
4	player_elo	0.062972
5	opening (grouped)	0.060825
6	self_mobility	0.039976
7	move_number	0.038754
8	branching_factor	0.034590
9	time_control_bucket (grouped)	0.020238

6. Phase 4 — Local Explanations

```
In [7]: for row_index in range(min(3, len(X_shap))):
        row = X_shap.iloc[[row_index]]
        explanation = explain_prediction(
            best_model,
            explainer,
            row,
            class_labels=LABEL_ORDER,
        )
        class_index = LABEL_ORDER.index(explanation['predicted_label'])
        print(f'--- example {row_index} ---')
        print(explanation)

        expected_value = explainer.expected_value
        if isinstance(expected_value, (list, np.ndarray)):
            base_value = expected_value[class_index]
```

```

else:
    base_value = expected_value

shap.plots.waterfall(
    shap.Explanation(
        values=shap_tensor[row_index, :, class_index],
        base_values=base_value,
        data=row.iloc[0].values,
        feature_names=list(X_shap.columns),
    ),
    max_display=12,
    show=False,
)
plt.tight_layout()
plt.show()

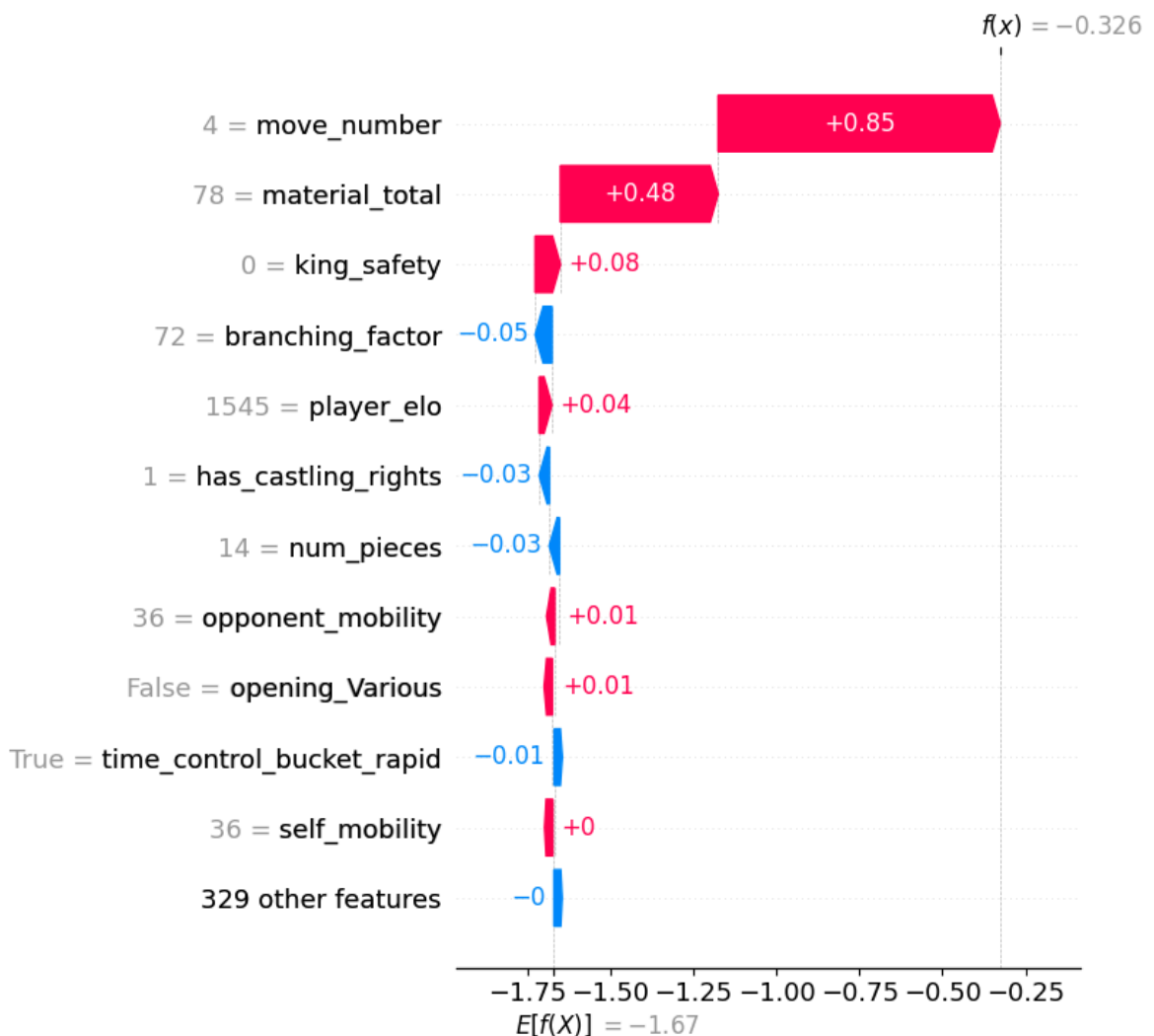
```

--- example 0 ---

```

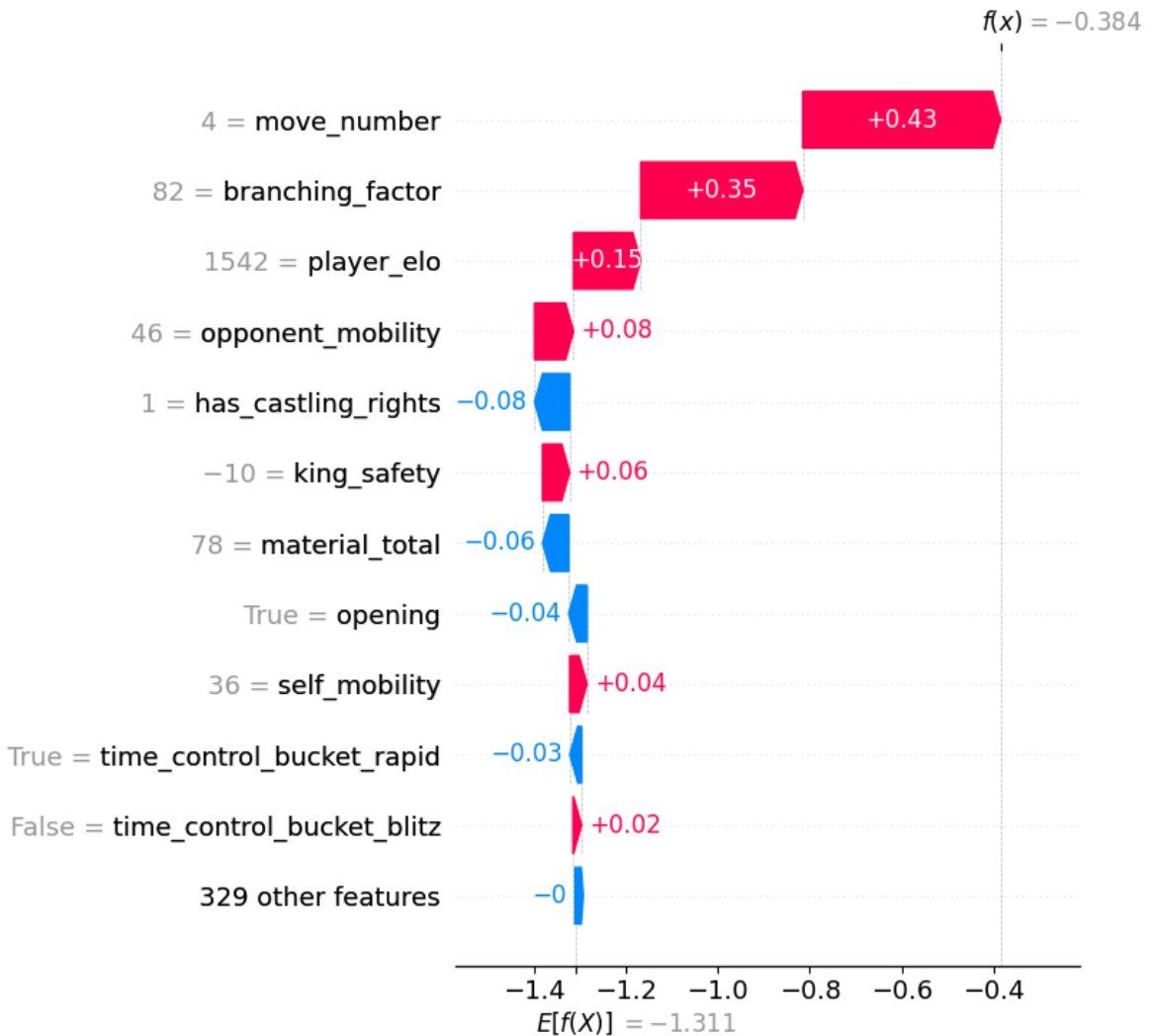
{'predicted_label': 'mistake', 'predicted_probabilities': {'good': 0.005901060725
9734746, 'inaccuracy': 0.2756473375735474, 'mistake': 0.5568095753952541, 'blunde
r': 0.16164202630522492}, 'top_positive_features': [{'feature': 'move_number', 'i
mpact': 0.8493835909722157}, {'feature': 'material_total', 'impact': 0.4750245046
982649}, {'feature': 'king_safety', 'impact': 0.07606469476348103}, {'feature':
'player_elo', 'impact': 0.03828105223148258}, {'feature': 'opponent_mobility', 'i
mpact': 0.013948080590412035}], 'top_negative_features': [{'feature': 'branching_
factor', 'impact': -0.05052135865012109}, {'feature': 'has_castling_rights', 'imp
act': -0.030236984896378084}, {'feature': 'num_pieces', 'impact': -0.030188002082
700463}, {'feature': 'time_control_bucket_rapid', 'impact': -0.00592596008919629
5}, {'feature': 'opening', 'impact': -0.0016251552158376124}]}

```



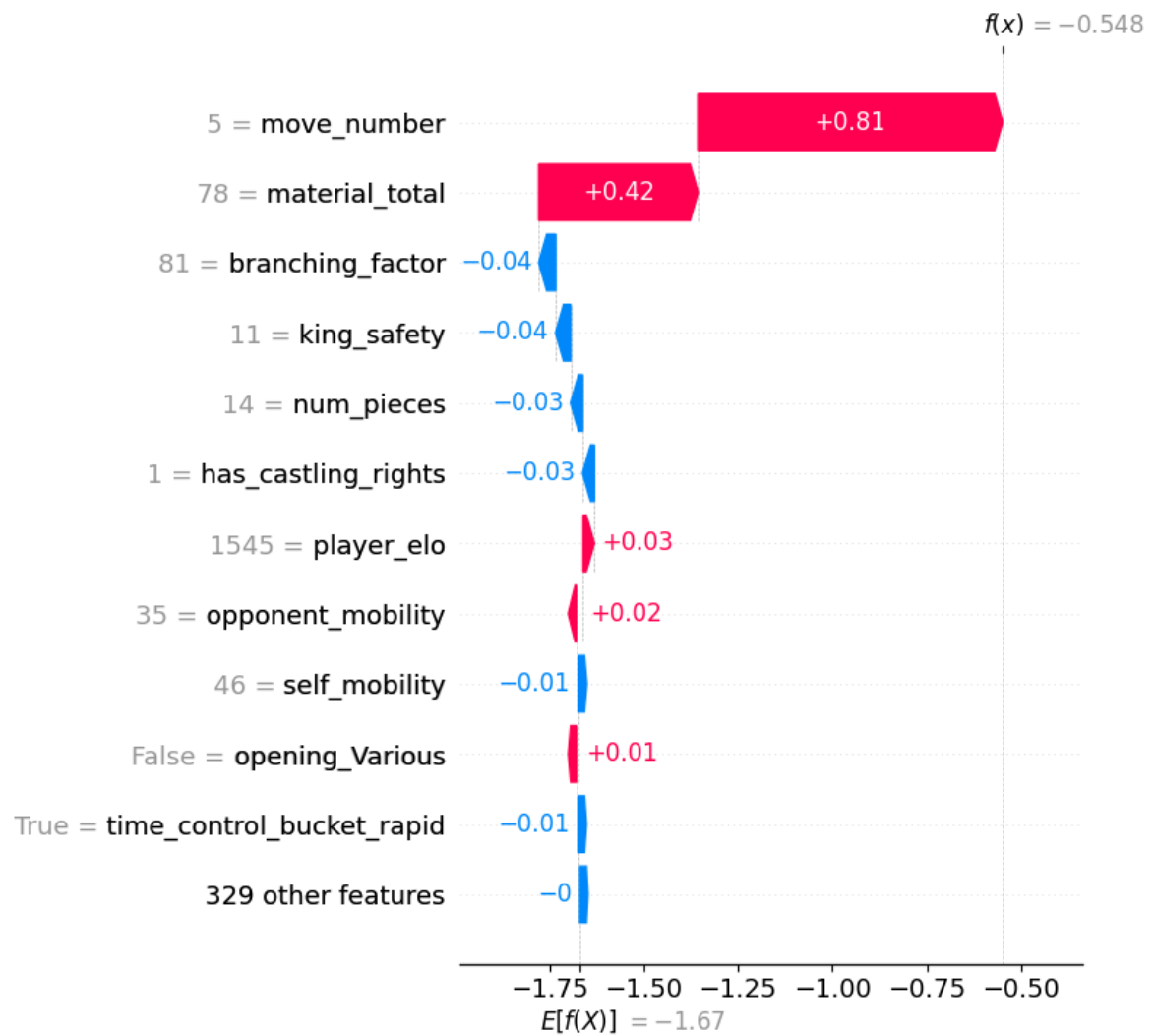
--- example 1 ---

```
{'predicted_label': 'inaccuracy', 'predicted_probabilities': {'good': 0.0051917208967668955, 'inaccuracy': 0.46828223912526684, 'mistake': 0.2984657335102256, 'blunder': 0.22806030646774084}, 'top_positive_features': [{'feature': 'move_number', 'impact': 0.43090332487219446}, {'feature': 'branching_factor', 'impact': 0.35326919123386447}, {'feature': 'player_elo', 'impact': 0.14647199576468437}, {'feature': 'opponent_mobility', 'impact': 0.08488067579659528}, {'feature': 'king_safety', 'impact': 0.058949241890789726}], 'top_negative_features': [{'feature': 'has_castling_rights', 'impact': -0.07643408992138727}, {'feature': 'material_total', 'impact': -0.05699423394921528}, {'feature': 'opening', 'impact': -0.03973628868711592}, {'feature': 'time_control_bucket_rapid', 'impact': -0.025558417087474174}, {'feature': 'opening_Various', 'impact': -0.014628777581510446}]}
```



--- example 2 ---

```
{'predicted_label': 'mistake', 'predicted_probabilities': {'good': 0.016152623728353962, 'inaccuracy': 0.19647727513593014, 'mistake': 0.5025961634446378, 'blunder': 0.2847739376910783}, 'top_positive_features': [{'feature': 'move_number', 'impact': 0.8080051523895957}, {'feature': 'material_total', 'impact': 0.42254997251746956}, {'feature': 'player_elo', 'impact': 0.029216897342119814}, {'feature': 'opponent_mobility', 'impact': 0.018294540265096946}, {'feature': 'opening_Various', 'impact': 0.00635288273157068}], 'top_negative_features': [{'feature': 'branching_factor', 'impact': -0.04462834451378288}, {'feature': 'king_safety', 'impact': -0.03991605810331316}, {'feature': 'num_pieces', 'impact': -0.03182233107094584}, {'feature': 'has_castling_rights', 'impact': -0.03072426645050623}, {'feature': 'self_mobility', 'impact': -0.0065375459703482085}]}
```



7. Conclusions

- SHAP is computed on the **best Human Pattern** model (not Proxy).
- Grouped opening importance avoids interpreting 300+ one-hot columns individually.
- Local impacts explain the model's predicted class, not Stockfish causality.
- Phases 5–7 (Pattern Engine, player profiles, coaching JSON) belong to later modules.

In []: